

INTRODUCTION A L'ALGORITHMIQUE ET A LA PROGRAMMATION

Dr Ibrahima GAYE - Université Alioune DIOP de Bambey

22 juin 2021

CHAPITRE III : LES STRUCTURES DE CONTRÔLE :
LES STRUCTURES ITERATIVES : les boucles

Table des matières

1	Introduction	4
2	LES STRUCTURES ITERATIVES	4
2.1	La boucle TantQue ... Faire	5
2.1.1	Fonctionnement de ce programme	6
2.1.2	La trace d'un algorithme	7
2.2	La boucle Pour	8
2.3	La boucle Faire ... TantQue	10
2.4	La boucle Répéter ... Jusqu'à	11
2.5	Différences entre la boucle Répéter, Faire TantQue et TantQue	13
2.6	La démarche itérative	14
2.6.1	Compter et Accumuler	14
2.6.2	La démarche itérative : cas général	19
2.7	L'imbrication des boucles	20
2.7.1	Exemple d'imbrication de boucles différentes	21
2.7.2	Application de la démarche descendante : les tables de multiplication	22

1 Introduction

- En programmation procédurale comme en algorithmique (qui respecte les contraintes fondamentales de la programmation!), **l'ordre des instructions est primordial.**
- Le processeur exécute les instructions dans l'ordre dans lequel elles apparaissent dans le programme. On dit que l'exécution est **séquentielle**.
- Une fois que le programme a fini une instruction, il passe à la suivante. Tant qu'une instruction n'est pas terminée, il attend avant de continuer. Par exemple, une instruction de saisie va attendre que l'utilisateur rentre une valeur au clavier avant de continuer.
- Parfois, il est nécessaire que le processeur n'exécute pas toutes les instructions, ou encore qu'il recommence plusieurs fois les mêmes instructions. Pour cela, il faudra casser la séquence. C'est le rôle des structures de contrôle.

Il existe deux grands types de structures de contrôle :

- les **structures conditionnelles** vont permettre de n'exécuter certaines instructions que sous certaines conditions
- les **structures répétitives**, encore appelées boucles, vont permettre de répéter des instructions un certain nombre de fois, sous certaines conditions

2 LES STRUCTURES ITERATIVES

Les structures répétitives aussi appelées boucles, permettent de **répéter un traitement (c'est à dire une instruction simple ou composée) autant de fois qu'il est nécessaire : soit un nombre déterminé de fois, soit tant qu'une condition est vraie.**

Il existe trois grands types principaux de structures répétitives :

- la structure **TantQue ... Faire**, qui permet d'effectuer une instruction tant qu'une condition est satisfaite.
- la structure **Faire ... Tantque**, une variante de la boucle **Tantque Faire** mais la condition est placée à la fin (boucle post-Contrôlée)
- la structure **Pour** qui permet de répéter une instruction un certain nombre de fois
- la structure **Répéter ... Jusqu'à**, qui, comme son nom l'indique, permet de répéter une instruction jusqu'à ce qu'une condition soit satisfaite.

Seule la boucle **Tantque** est fondamentale. Avec cette boucle, on peut réaliser toutes les autres boucles alors que l'inverse n'est pas vrai. La boucle **Pour** est très utilisée aussi car elle permet de simplifier la boucle **TantQue** lorsque le nombre de tour de boucle est connu d'avance. La boucle

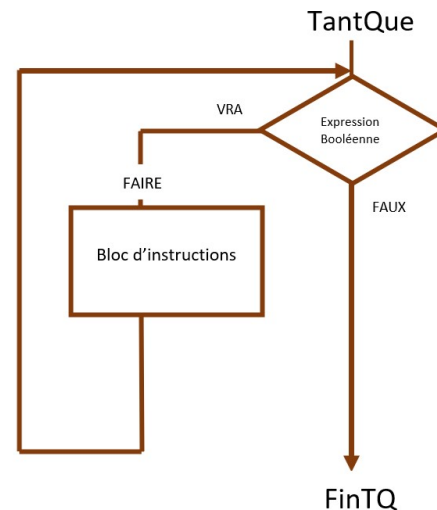
Répéter et la boucle **Faire TantQue** sont des variantes de la boucle **TantQue** ; leur condition est évaluée à la fin

2.1 La boucle TantQue ... Faire

La boucle **TantQue ... Faire** permet de répéter un traitement tant qu'une expression conditionnelle est vraie. **Si d'emblée, la condition n'est pas vraie, le traitement ne sera pas exécuté.** On voit donc que la boucle Tant que a un point commun avec la structure conditionnelle stricte où si la condition n'est pas vraie, le traitement n'est pas exécuté.

Ci-après, on a la syntaxe et l'algonigramme d'une d'une boucle TantQue ... Faire :

TantQue (Condition) Faire
 Bloc d'instructions
FIN TantQue



Supposons que l'on veuille que l'algorithme calcule le cube des nombres qu'on lui fournit et que pour arrêter, l'utilisateur doive entrer 0. Si le nombre saisi est 0, on ne va pas afficher le cube et le traitement est terminé. Si le nombre saisi est différent de 0, on affiche son cube et on recommence (on demande d'entrer un nombre, on le saisit, etc.)

On veut donc exécuter les instructions dans l'ordre suivant :

- Lire un nombre
- vérifier la condition d'exécution ($x \neq 0$)
- si x vaut 0, on sort de la boucle sinon on affiche le cube et on attend que l'utilisateur entre un autre nombre
- On vérifie la condition d'exécution ($x \neq 0$)
- si x vaut 0, on sort de la boucle sinon on affiche le cube et on attend que l'utilisateur entre un autre nombre ...

On voit donc qu'après la saisie du premier nombre, on répète les trois dernières instructions. On va

donc pouvoir les inscrire dans une boucle. La condition de continuation ($x \neq 0$) est inscrite après le tant que. Cette condition est vérifiée à chaque fois qu'on a terminé les traitements de la boucle.

Algorithme cube**Variables**

x : entier

Début

Afficher "Ce programme calcul le cube des nombres que vous entrez. Pour arrêter tapez
0"

Afficher "Entrez un nombre"

Lire x

TantQue ($x \neq 0$) Faire

Afficher "le cube de " , x , " est " , $x*x*x$

Afficher "Entrez un nombre ou 0 pour arrêter"

Lire x

Fin tantQue Afficher "Fin des itérations"

Fin

Le nombre de répétition du traitement n'est pas indiqué explicitement ; il dépendra des données fournies au programme, en l'occurrence les nombres entrés.

2.1.1 Fonctionnement de ce programme

Il affiche tout d'abord le libellé de saisie et attend que l'utilisateur entre un nombre, qui est alors saisi dans la variable x. Ensuite, la condition qui suit le Tant que est évaluée. Si l'utilisateur rentre comme premier nombre 0, la condition est fausse et le corps de la boucle ne sera pas exécuté et le processeur continuera à la première instruction suivant le **Fin tantque** (Afficher "Fin des itérations"). Si l'utilisateur entre un nombre différent de 0, son cube est calculé et affiché et un nouveau nombre est saisi. Au niveau du **Fin tantque**, le processeur effectue un branchement, c'est-à-dire qu'il n'effectue pas l'instruction suivante mais retourne au début de la boucle et réévalue l'expression conditionnelle.

L'utilisateur peut calculer autant de cubes qu'il désire et quand il veut arrêter, il lui suffit de taper 0. On dit que 0 est une valeur drapeau, c'est-à-dire une valeur qui indique la fin d'un traitement.

2.1.2 La trace d'un algorithme

La trace d'un algorithme représente la valeur des différentes informations d'un programme durant son exécution. Il est vivement conseillé d'effectuer la trace d'un algorithme afin de vérifier qu'il fonctionne. La première chose à faire est de choisir des données sur lesquelles on va effectuer le test de l'algorithme. Pour ces données, on calcule à la main le résultat attendu. Puis on effectue la trace et on compare le résultat attendu avec le résultat de la trace qui doivent être les mêmes (sinon, il y a une erreur quelque part...)

Effectuons la trace de l'algorithme précédent avec les données suivantes :

donnée x	résultat attendu
10	affichage de 1000
-3	affichage de -27
4	affichage de 16
0	affichage de Fin et arrêt du programme

Instruction exécutée	Variable expression évaluée	Valeur après l'instruction	Affichage à l'écran
Afficher "Ce programme calcul le cube des nombres que vous entrez.Pour arrêter tapez 0"			Ce programme calcul le cube des nombres que vous entrez.Pour arrêter tapez 0
Afficher "Entrez un nombre"			Entrez un nombre
Lire x	x	10	
TantQue $x \neq 0$	$x \neq 0$	VRAI	
Afficher Le cube de "x," est " $x*x*x$ "	$x*x*x$	1000	Le cube de 10 est 1000
Afficher "Entrez un nombre" ou 0 pour arrêter"			Entrez un nombre ou 0 pour arrêter
Lire x	x	-3	
TantQue $x \neq 0$	$x \neq 0$	VRAI	
Afficher Le cube de "x," est " $x*x*x$ "	$x*x*x$	-27	Le cube de -3 est -27
Afficher "Entrez un nombre" ou 0 pour arrêter"			Entrez un nombre ou 0 pour arrêter
Lire x	x	4	
TantQue $x \neq 0$	$x \neq 0$	VRAI	
Afficher Le cube de "x," est " $x*x*x$ "	$x*x*x$	64	Le cube de 4 est 64
Afficher "Entrez un nombre" ou 0 pour arrêter"			Entrez un nombre ou 0 pour arrêter
Lire x	x	0	
TantQue $x \neq 0$	$x \neq 0$	FAUX	
Afficher "Fin des itérations"			Fin des itérations

2.2 La boucle Pour

La boucle Pour permet de répéter une instruction **un nombre connu de fois**. Elle a le formalisme suivant :

Pour <compteur> **de** <valeur initiale> **à** <valeur finale> [**pas de** <incrément>] **Faire**
 <traitement>

FinPour

ou

Pour <compteur> **de** <valeur initiale> **à** <valeur finale> [**pas de** <décrément>] **Faire**
 <traitement>

FinPour

Elle permet de faire la même chose que la boucle **Tantque** mais de façon plus rapide, du moins lorsque le nombre de répétition est connu.

La variable compteur est de type entier. Elle est initialisée à la valeur initiale. **Le compteur augmente (respectivement diminue) (implicitement) de l'incrément (respectivement du décrément) à chaque répétition du traitement.** Lorsque la variable compteur vaut la valeur finale, le traitement est exécuté une dernière fois puis le programme sort de la boucle.

↗ Exemple :

Pour x **de** 1 **à** 20 **Faire**

 Afficher x, "*", x, "=", x*x

FinPour

Grâce à une telle structure, le traitement va être répété 20 fois. On pourrait faire la même chose avec une boucle tant que, mais il faudrait initialiser la variable compteur et l'incrémenter explicitement.

x ← 1

TantQue x ≤ 20 **Faire**

 Afficher x, "*", x, "=", x*x

 x ← x+1 ► incrémentation automatique dans la boucle for

FinTantQue

La boucle Pour est en fait une simplification de la boucle TantQue.

↗ Application :

Affichons la table de multiplication du 7. Pour cela on va utiliser une variable a qui varie de 1 à 10 et multiplier cette variable par 7 à chaque incrémentation. Cette variable va aussi servir de compteur pour la structure Pour.

Algorithme Table7**Variables**

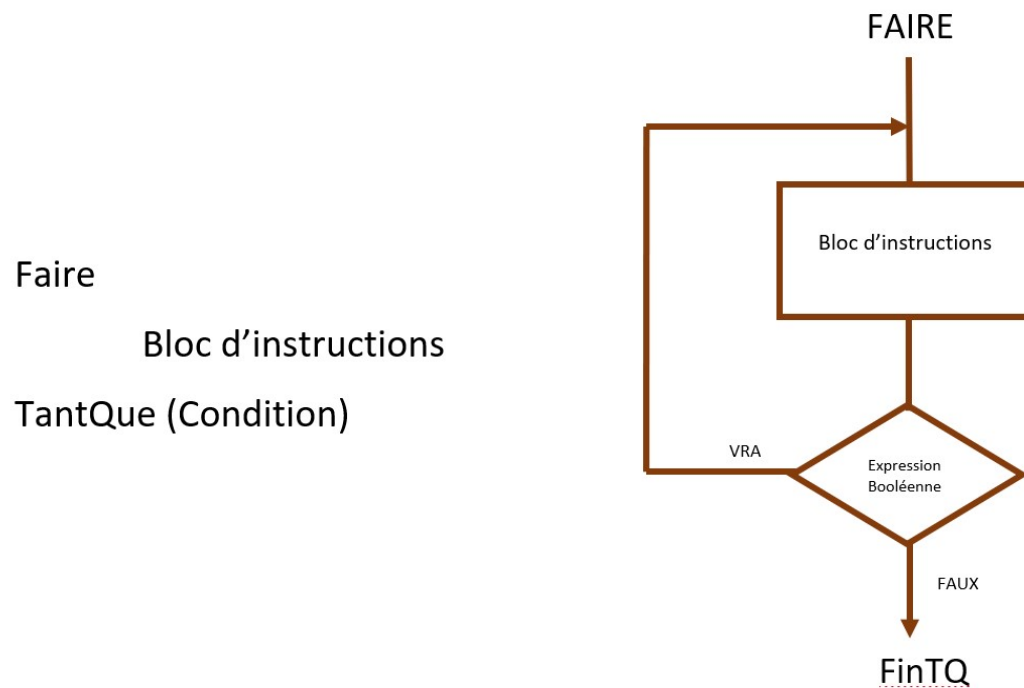
a : entier

Début**Pour a de 1 à 10 pas de 1**

Afficher a, " * 7 = ", a * 7

FinPour**Fin****2.3 La boucle Faire ... TantQue**

Cette boucle sert à répéter une instruction jusqu'à ce qu'une condition (expression booléenne) soit Fausse. Son formalisme est le suivant :



Le traitement est exécuté, puis la condition est vérifiée. Si elle n'est pas Vraie, on retourne au début de la boucle et le traitement est répété. Si la condition est Fausse, on sort de la boucle et le programme continue séquentiellement. A chaque fois que le traitement est exécuté, la condition d'exécution est de nouveau vérifiée à la fin.

Exemple : ALGORITHME Aire

VARIABLES

rayon : réel

reponse : caractere

DEBUT

Afficher "Cet algorithme calcul l'aire d'un cercle"

Faire

Afficher "Entrer le rayon du cercle"

Lire rayon

Afficher "L'aire de ce cercle est :", $\text{rayon} * \text{rayon} * 3.14$

Afficher "Voulez vous calculer l'aire d'un autre cercle? (Oui/Non)"

Lire reponse

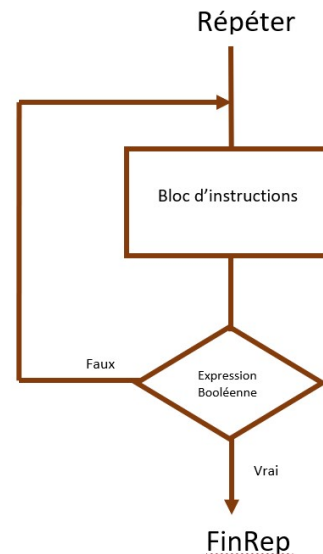
TantQue reponse = Oui

FIN

2.4 La boucle Répéter ... Jusqu'à

Cette boucle sert à répéter une instruction jusqu'à ce qu'une condition (expression booléenne) soit vraie. Son formalisme est le suivant :

Répéter
Bloc d'instructions
Jusqu'à (Condition)



Le traitement est exécuté, puis la condition est vérifiée. Si elle n'est pas Fausse, on retourne au début de la boucle et le traitement est répété. Si la condition est vraie, on sort de la boucle et le

programme continue séquentiellement. A chaque fois que le traitement est exécuté, la condition d'arrêt est de nouveau vérifiée à la fin.

Exemple : ALGORITHME Aire

VARIABLES

rayon : réel

reponse : caractere

DEBUT

Afficher "Cet algorithme calcul l'aire d'un cercle"

REPETER

Afficher "Entrer le rayon du cercle"

Lire rayon

Afficher "L'aire de ce cercle est :", $\text{rayon} * \text{rayon} * 3.14$

Afficher "Voulez vous calculer l'aire d'un autre cercle? (Oui/Non)"

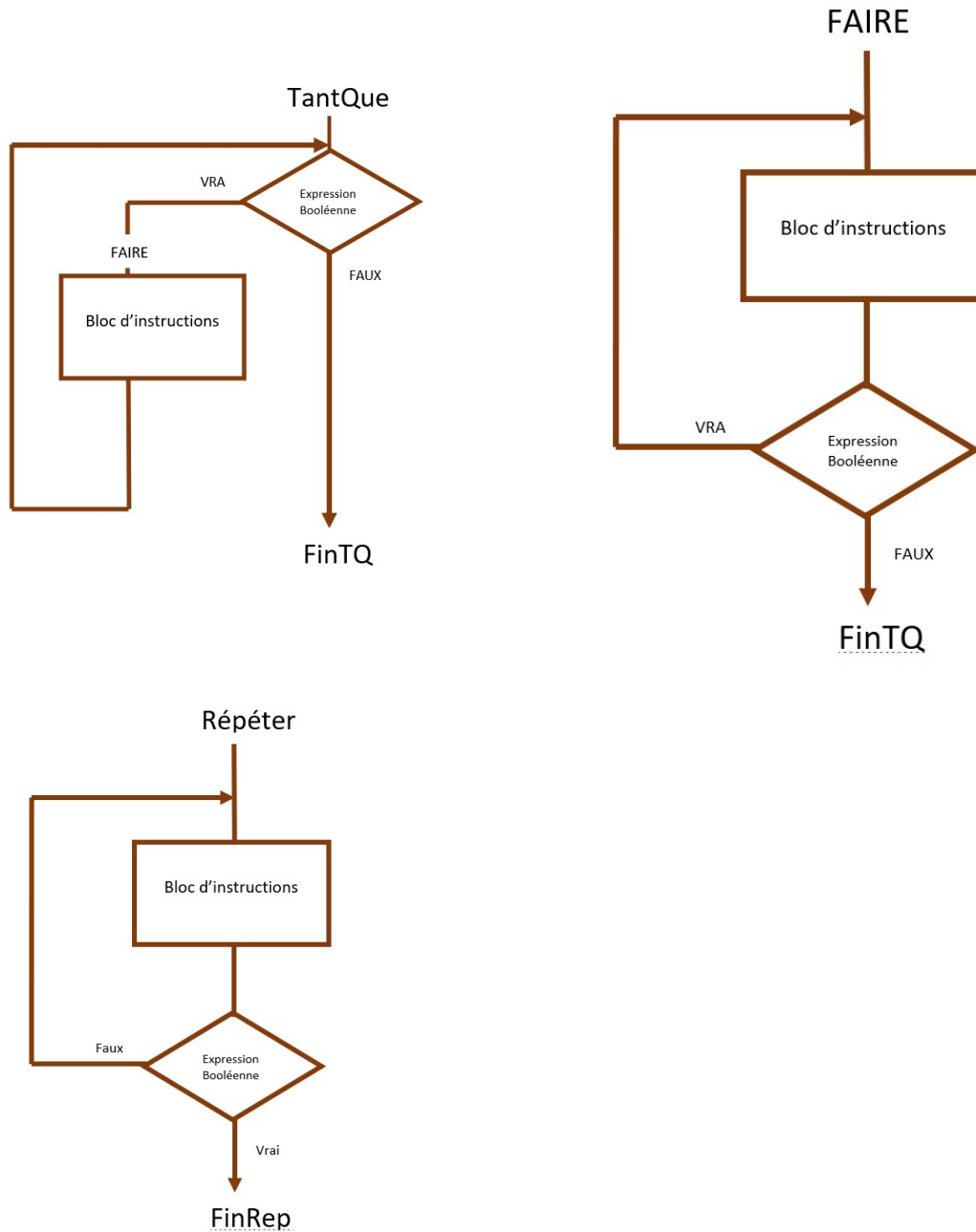
Lire reponse

JUSQU'A reponse $\neq$ Oui

FIN

La boucle Répéter n'est pas indispensable. Elle peut toujours être remplacée par une boucle **TantQue** ou **Faire ... TantQue**. C'est pourquoi certains langages n'ont pas d'équivalent pour la boucle **Répéter**.

2.5 Différences entre la boucle Répéter, Faire TantQue et TantQue



TantQue ... Faire

- La condition est vérifiée avant le traitement
- Le traitement peut ne pas se faire
- Vous avez une **condition d'exécution (de continuation)**
 - Si la condition est vraie, alors le corps de la boucle est exécuté

Répéter ... Jusqu'à

- La condition est vérifiée après le traitement
- Le traitement va se faire au moins une fois
- Vous avez une **condition d'arrêt**
 - Si la condition est fausse, alors le corps de la boucle est ré-exécuté

Faire ... TantQue

- La condition est vérifiée après le traitement
- Le traitement va se faire au moins une fois
- Vous avez une **condition d'exécution**
 - Si la condition est vraie, alors le corps de la boucle est ré-exécuté

2.6 La démarche itérative

Une itération est une boucle où la valeur d'une variable dépend de sa valeur au tour précédent. La variable en question se trouve à la fois à gauche et à droite d'une affectation.

La démarche itérative (l'utilisation d'itérations) est utilisée pour résoudre beaucoup de problèmes de programmation. Pour se familiariser avec cette démarche complexe, nous allons étudier des problèmes algorithmiques simples et fondamentaux.

2.6.1 Compter et Accumuler

Comment faire pour compter le nombre de tour de boucle dans une boucle Tantque ? (comptage systématique)

Reprenons le programme cube. Supposons maintenant que nous ayons besoin de compter combien de cubes ont été calculés. Comment procéder ?

Il suffit d'utiliser une variable qui va servir de compteur. Avant l'entrée dans la boucle, le compteur est mis à 0. Ce compteur est incrémenté de 1 à chaque tour de boucle. Pour cela, on ajoute l'instruction **compteur** $\leftarrow$ **compteur** + 1 à l'intérieur de la boucle : une telle instruction s'appelle **incréméntation**.

Algorithme cube

Variables

x : entier

compteur : entier

Début

Afficher "Ce programme calcul le cube des nombres que vous entrez. Pour arrêter tapez 0."

Afficher "Entrez un nombre"

Lire x

Tantque $x \neq 0$ Faire

Afficher "le cube de " , x , " est " , $x*x*x$

compteur $\leftarrow$ compteur + 1

Afficher "Entrez un nombre ou 0 pour arrêter"

Lire x

FinTantque

Afficher "Vous avez demandé", compteur, "cubes"

Fin

Comment faire pour compter seulement les cubes négatifs ? (comptage sélectif)

Si on ne veut augmenter le compteur que dans une certaine condition (ici, dans le cas où le nombre saisi est négatif), il suffit de placer l'incrémentation à l'intérieur d'une structure conditionnelle.

Algorithme cubeNégatif Variables

x : entier

compteur : entier

Début

Afficher "Ce programme calcul le cube des nombres que vous entrez. Pour arrêter tapez 0."

Afficher "Entrez un nombre"

Lire x

compteur $\leftarrow$ 0

Tantque x $\neq$ 0 **Faire**

Si x < 0 **alors**

 Afficher "le cube de " , x , " est " , x*x*x

 compteur $\leftarrow$ compteur + 1

FinSi

 Afficher "Entrez un nombre ou 0 pour arrêter"

 Lire x

FinTantque

Afficher "Vous avez demandé", compteur, "cubes négatifs"

Fin**Comment faire pour compter plusieurs choses à la fois ? (comptage multiple)**

On peut vouloir compter plusieurs choses simultanément dans la même boucle. Pour reprendre notre exemple, nous pourrions vouloir compter les cubes négatifs mais aussi les cubes pairs. Dans ce cas, un seul compteur ne suffit plus. Il faut utiliser autant de compteur que l'on a de choses à compter.

Algorithme cubeMultiple**Variables**

x : entier

cptNegatif : entier ► Compteur des cubes négatifs**cptPair** : entier ► Compteur des cubes pairs**Début**

Afficher "Ce programme calcul le cube des nombres que vous entrez. Pour arrêter tapez
0."

Afficher "Entrez un nombre"

Lire x

cptNegatif $\leftarrow$ 0cptPair $\leftarrow$ 0**Tantque** x $\neq$ 0 **Faire**Afficher "le cube de " , x , " est " , $x*x*x$ **Si** ($x*x*x$) < 0 **alors**cptNegatif $\leftarrow$ cptNegatif + 1**FinSi****Si** ($x*x*x$) mod 2=0 **alors**cptPair $\leftarrow$ cptPair + 1**FinSi**

Afficher "Entrez un nombre ou 0 pour arrêter"

Lire x

FinTantque

Afficher "Vous avez obtenu", cptNegatif, "cubes négatifs, et", cptPair, "cubes pairs"

Fin**Calculer le résultat de x^n avec une itération**

Dans certains langages, l'opérateur exposant n'existe pas. Supposons que nous ne pouvons pas l'utiliser en algorithmique. Nous allons écrire l'algorithme qui permet de calculer un nombre à un exposant donné. Le nombre x et l'exposant n sont saisis.

Rappel :

$$x^1 = x$$

$$x^2 = x * x = x^1 * x$$

$$x^3 = x * x * x = x^2 * x$$

$$x^4 = x * x * x * x = x^3 * x$$

...

On ne peut pas faire tout d'un coup le nombre de multiplication nécessaire car on ne sait pas combien vaut l'exposant au moment d'écrire le programme. Le programmeur ne sait pas quel exposant va taper l'utilisateur. Il y a une infinité de possibilités.

Pour contourner cette difficulté, on va répéter n fois la multiplication par x dans une boucle.

On utilise la boucle Pour car on sait combien de fois on répète la multiplication : n fois.

Que fait-on du résultat de la multiplication par : on l'affecte dans une variable résultat, que l'on va utiliser au tour suivant. Qu'est-ce qu'on multiplie par x à chaque tour : le résultat du tour précédent.

Et au premier tour ? Il n'y a pas encore de résultat. Il suffit d'initialiser la variable résultat avec 1.

D'où la solution suivante :

Algorithme exposant

Variables

x, n : entier ► x le nombre et n l'exposant

Début

Afficher "veuillez entrez un nombre puis son exposant"

Lire x, n

$res \leftarrow 1$ ► initialisation du résultat

Pour i de 1 à n **Faire**

$res \leftarrow res * x$ ► itération

FinPour

Afficher x , "puissance", n , "vaut", res

Fin

La variable résultat (res) ne vaut véritablement le résultat recherché qu'à la sortie de la boucle. Entre temps, elle prend des valeurs intermédiaires qui servent à avancer d'une valeur initiale connue vers la valeur finale recherchée.

Exercice : Effectuer la trace du morceau d'algorithme dans le cas où l'utilisateur entre 5 pour x et 4 pour n .

Trouver le minimum d'une suite de nombres

Nous voulons trouver le plus petit parmi une liste de 100 nombres saisis par l'utilisateur.

Comment faire ?

Il est irréaliste de déclarer 100 variables et de les comparer toutes une à une. La saisie des nombres de la liste va se faire à l'intérieur d'une boucle à l'aide d'une seule variable.

Pour obtenir le minimum, nous allons utiliser une itération. A chaque tour de boucle, un nombre supplémentaire est saisi. Si on connaît le minimum parmi tous les précédents nombres saisis, il suffit de comparer le nouveau nombre à ce minimum pour avoir le nouveau minimum (parmi tous les nombres, y compris le dernier).

Si le nouveau nombre saisi est plus petit que le plus petit des nombres précédents, alors le nouveau nombre est le nouveau minimum parmi tous les nombres saisis. Sinon, le minimum reste le même.

Avant la première saisie, il n'y a pas de minimum. En fait, lorsqu'un seul nombre est saisi, c'est forcément lui le minimum. Donc on commence à faire la boucle à partir du deuxième élément saisi.

Algorithme minimum

Variables

nb : réel ► pour la saisie des nombres

min : réel ► minimum

Début Afficher "Entrez un nombre"

Lire nb

min nb ► le premier nombre saisi est le minimum des nombres déjà saisi car seul

Pour i de 2 à 100 Faire ► pour tous les autres éléments du 2 au dernier

Afficher "entrez un autre nombre"

Lire nb ► on le met en mémoire

Si nb < min **Alors** ► si il est plus petit que le minimum trouvé au tour précédent

min nb ► c'est lui le nouveau minimum parmi les nombre déjà saisis

Finsi

Finpour ► à la sortie de la boucle, min vaut le minimum des 100 nombres saisis

Afficher "Le minimum des nombres saisis est", min **Fin**

2.6.2 La démarche itérative : cas général

Une itération consiste à un cheminement d'un état initial à un état final, celui qui est recherché.

Un état est représenté par les valeurs des variables à un moment donné. La progression (le chemi-

nement) vers l'état recherché se fait en passant par des états intermédiaires. Une boucle permet de progresser d'un état à un autre état, en se rapprochant de l'état final. Lorsque l'état final est atteint, la boucle doit d'arrêter.

NB : Pour "découvrir" une itération, il n'y a pas de recette miracle. Il faut utiliser son imagination et son intelligence. Néanmoins, la démarche suivante peut aider à trouver une itération pour résoudre un problème.

- Chercher un état intermédiaire entre l'état initial et l'état final (par exemple, pour le minimum étudié au paragraphe précédent, l'état intermédiaire est qu'on a trouvé le minimum des i nombres déjà tapés). On ne se préoccupe pas de la manière dont on est parvenu à cet état, on suppose que cet état est atteint.
- Voir quelles instructions permettent de progresser à l'état intermédiaire suivant (par exemple, pour le minimum, comment trouver le nouveau minimum lorsqu'un nouveau nombre est tapé). Bien faire attention à l'ordre des instructions.
- Se demander à quelle(s) condition(s) l'état auquel on est parvenu est final (dans le cas du minimum, c'est lorsque tous les nombres ont été saisis, c'est-à-dire quand le compteur vaut plus de 100)
- Enfin, trouver comment commencer. Il faut trouver un état initial où toutes les valeurs sont connues et qui permet de passer à un état intermédiaire. Certaines variables doivent être initialisées. (dans le cas du minimum, le minimum est connu quand un seul nombre a été saisi)

2.7 L'imbrication des boucles

Comme les structures conditionnelles, les boucles peuvent être imbriquées. Cette partie n'apporte pas de notion nouvelle mais permet de souligner certains aspects délicats.

Le morceau d'algorithme suivant permet simplement d'afficher 5 étoiles. Il n'y a pas d'imbrication pour l'instant.

```
POUR i DE 1 JUSQU'A 5 FAIRE
```

```
    Afficher "*" "
```

```
FinPOUR
```

Si on inclut cette séquence à l'intérieur d'une boucle qui se répète 3 fois, on va effectuer 3 fois l'affichage de 5 étoiles donc on va obtenir l'affichage de 15 étoiles. L'imbrication des boucles multiplie les tours de boucles.

```
POUR j DE 1 JUSQU'A 3 FAIRE
```

```
    POUR i DE 1 JUSQU'A 5 FAIRE
```

```
        Afficher "*" "  
    FinPOUR  
FinPOUR
```

Remarque TRES importante : Lorsque l'on imbrique des boucles, IL FAUT UTILISER DES COMPTEURS DIFFÉRENTS pour chaque boucle.

Pour pouvoir afficher 3 lignes de 5 étoiles, au lieu de 15 étoiles à la suite, il suffit d'ajouter un saut de ligne après le Pour le plus imbriqué.

```
POUR j DE 1 JUSQU'A 3 FAIRE  
    POUR i DE 1 JUSQU'A 5 FAIRE  
        Afficher "*" "  
    FinPOUR  
    Afficher "\n"
```

```
FinPOUR
```

La boucle la plus imbriquée s'exécute alors que la "grande" boucle s'exécute seulement 3 fois. Le compteur i varie 5 fois plus vite que le compteur j.

Tous les types de boucles peuvent s'imbriquer entre eux. **La seule règle à respecter est que les boucles ne doivent pas se chevaucher : elles doivent s'emboîter.** Si vous respectez bien la règle des décalages, vous ne pouvez pas vous tromper.

2.7.1 Exemple d'imbrication de boucles différentes

Voilà un programme qui calcule le chiffre d'affaire annuel d'un représentant à partir de la saisie des 12 chiffres d'affaire mensuels. Le programme permet de recommencer le calcul avec un autre représentant si l'utilisateur tape le caractère 'o' (pour oui).

ALGORITHME CiffreAfaire VARIABLES

```
CAM : réel ► Chiffre d'affaire mensuel  
CAA : réel ► Chiffre d'affaire annuel  
mois : entier ► Compteur de la boucle qui prend les différents mois  
reponse : caractere
```

DEBUT

```
    REPETER
```

```
Afficher "Entrer les 12 chiffres d'affaire mensuels d'un représentant"
Lire CAA ← 0 ► Initialisation du chiffre d'affaire annuel
POUR mois DE 1 JUSQU'A 12 FAIRE
    Lire CAM
    CAA ← CAM+CAA ► Cumul
FinPOUR
Afficher "Chiffre d'affaire annuel : ",CAA
Afficher "Voulez-vous calculer le chiffre d'affaire d'un autre représentant ?
(oui/non)"
Lire reponse
JUSQU'À reponse=non
```

FIN

L'imbrication des boucles n'est pas compliquée si on pense à bien décomposer les problèmes, du général au particulier (c'est **l'approche descendante**). Pour cela, il faut procéder à une **analyse** du problème sans rentrer dans les détails de l'algorithme. On réfléchit d'abord à **QUOI** faire avant de réfléchir à **COMMENT** faire. Le quoi faire de l'algorithme précédent pourrait s'exprimer ainsi.

REPETER

```
Calculer le chiffre d'affaire d'un représentant
Demander l'utilisateur s'il veut continuer
```

JUSQU'A ce que la réponse de l'utilisateur est non

A l'issue de cette étape d'analyse, on réfléchit au COMMENT. On se rend compte alors que pour calculer le chiffre d'affaire d'un représentant, il faut utiliser une boucle Pour. Cette boucle vient donc naturellement s'imbriquer dans la boucle Répéter.

2.7.2 Application de la démarche descendante : les tables de multiplication

On veut écrire un algorithme qui calcule et affiche les 10 premières tables de multiplication (de 1 à 10).

Chaque table doit être présentée comme exemple de la table du 7 ci-dessous : Table : 7

$$7 * 1 = 7$$

$$7 * 2 = 14$$

7 * 3= 21
7 * 4= 28
7 * 5= 35
7 * 6= 42
7 * 7= 49
7 * 8= 56
7 * 9= 63
7 * 10=70

Plutôt que d'essayer d'écrire immédiatement l'algorithme complet, il est préférable de faire une approche descendante du problème. Cela consiste à décomposer le problème en sous-problèmes plus simples à résoudre. Si certains sous-problèmes sont décomposables en problèmes plus petits, on les décompose encore. Ensuite, chaque sous-problème est résolu séparément et enfin, ils sont rassemblés pour composer la solution complète.

Dans notre problème, nous constatons qu'il s'agit d'afficher 10 tables de multiplication, le numéro de la table augmentant d'un à chaque fois : on va donc utiliser une boucle Pour.

Le traitement décrit en italique doit être précisé. Pour écrire la table des n, on affiche d'abord le libellé, puis on utilise une boucle pour chaque nombre de 1 à 10. **ATTENTION** : n est le numéro de la table, donc on ne peut pas l'utiliser comme compteur dans la boucle des 10 nombres. On va plutôt utiliser l comme ligne.

```
Afficher "Table : ",n
POUR i DE 1 JUSQU'A 10 FAIRE
    Afficher n," * ",i," = ",n*i
FinPOUR
```

On va maintenant recomposer le programme entier en ajoutant les déclarations habituelles : POUR n DE 1 JUSQU'A 10 FAIRE

```
Afficher "Table : ",n
POUR i DE 1 JUSQU'A 10 FAIRE
    Afficher n," * ",i," = ",n*i
FinPOUR
FinPOUR
```